

Actividad UT 5 – Spring



Índice

Enunciado	3
Explicación del código	4
Capturas de pantalla	13
Código completo	19
Acceso al zip del proyecto	27

Enunciado

Actividad UT 5 - Spring

Enunciado

La Mesa Redonda necesita un sistema para gestionar a los guerreros Sinluz que recorren las Tierras Intermedias.

Cada personaje tiene atributos básicos y puede evolucionar a lo largo de su aventura.

Se pide desarrollar una aplicación que permita registrar, consultar y modificar personajes, aplicando ciertas reglas de negocio desde la capa de servicio.

Funcionalidades obligatorias

La aplicación deberá permitir:

1. Crear un personaje
 - Introducir nombre, clase y nivel inicial
 - El nivel inicial no puede ser menor que 1
2. Listar todos los personajes
 - Mostrar ID, nombre, clase y nivel
3. Subir de nivel a un personaje
 - Se selecciona el personaje por ID
 - El nivel aumenta en una cantidad indicada por el usuario
 - El nivel máximo permitido es 100
4. Cambiar la clase de un personaje
 - Se selecciona el personaje por ID
 - Se asigna una nueva clase
5. Eliminar un personaje caído
 - Se borra un personaje por ID
6. Salir de la aplicación

Modelo de datos

Se deberá crear una entidad llamada Personaje con los siguientes atributos:

- id (Long, clave primaria, autogenerada)
- nombre (String)
- clase (String)
- nivel (int)

Menú por consola

El menú deberá mostrarse repetidamente hasta que el usuario decida salir, por ejemplo:

--- MESA REDONDA ---

1. Crear personaje
2. Listar personajes
3. Subir nivel
4. Cambiar clase
5. Eliminar personaje
0. Abandonar las Tierras Intermedias

Explicación del código

CrudmesaApplication.java:

```
package com.example.crudmesa;

import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Bean;

import java.util.Scanner;

@SpringBootApplication
public class CrudmesaApplication {

    public static void main(String[] args) {
        SpringApplication.run(CrudmesaApplication.class, args);
    }

    @Bean
    CommandLineRunner ejecutar(PersonajeService service) {

        return args -> {

            Scanner sc = new Scanner(System.in);
            int opcion;

            do {
                System.out.println("\n--- MESA REDONDA ---");
                System.out.println("1. Crear personaje");
                System.out.println("2. Listar personajes");
                System.out.println("3. Subir nivel");
                System.out.println("4. Cambiar clase");
                System.out.println("5. Eliminar personaje");
                System.out.println("0. Abandonar las Tierras Intermedias");
                System.out.print("Opción: ");

                opcion = sc.nextInt();
                sc.nextLine();

                switch (opcion) {
```

```

        case 1 -> {
            System.out.print("Nombre: ");
            String nombre = sc.nextLine();

            System.out.print("Clase: ");
            String clase = sc.nextLine();

            System.out.print("Nivel inicial: ");
            int nivel = sc.nextInt();
            sc.nextLine();

            try {
                service.crear(new Personaje(nombre, clase,
nivel));

                System.out.println("Personaje creado
correctamente");
            } catch (Exception e) {
                System.out.println("Error: " +
e.getMessage());
            }
        }

        case 2 -> {
            System.out.println("\nLISTADO:");
            service.listar().forEach(p ->
                System.out.println(
                    p.getId() + " - " +
                    p.getNombre() + " - " +
                    p.getClase() + " - Nivel " +
                    p.getNivel()
                )
            );
        }

        case 3 -> {
            System.out.print("ID del personaje: ");
            Long id = sc.nextLong();

            System.out.print("Subir niveles: ");
            int subida = sc.nextInt();
            sc.nextLine();

```

```

        try {
            service.subirNivel(id, subida);
            System.out.println("Nivel actualizado");
        } catch (Exception e) {
            System.out.println("Error: " +
e.getMessage());
        }
    }

    case 4 -> {
        System.out.print("ID del personaje: ");
        Long id = sc.nextLong();
        sc.nextLine();

        System.out.print("Nueva clase: ");
        String nuevaClase = sc.nextLine();

        try {
            service.cambiarClase(id, nuevaClase);
            System.out.println("Clase cambiada");
        } catch (Exception e) {
            System.out.println("Error: " +
e.getMessage());
        }
    }

    case 5 -> {
        System.out.print("ID del personaje: ");
        Long id = sc.nextLong();
        sc.nextLine();

        service.borrar(id);
        System.out.println("Personaje eliminado");
    }
}

} while (opcion != 0);

System.out.println("Has abandonado las Tierras
Intermedias...");
};
}
}

```

CrudmesaApplication.java es la clase principal del proyecto y representa el punto de entrada de la aplicación. Está anotada con `@SpringBootApplication`, lo que indica que es una aplicación Spring Boot y que debe activarse la configuración automática, el escaneo de componentes y la gestión del contexto de Spring. En el método `main` se llama a `SpringApplication.run`, que arranca el contenedor de Spring, crea todos los beans necesarios y deja la aplicación preparada para funcionar.

Dentro de esta clase también se define un método anotado con `@Bean` que devuelve un `CommandLineRunner`. Este tipo de componente permite ejecutar código una vez que Spring ha terminado de inicializar todos sus objetos. Es importante porque el menú necesita que el servicio `PersonajeService` ya esté creado e inyectado por Spring. El propio método recibe como parámetro el servicio, y Spring lo proporciona automáticamente gracias a la inyección de dependencias.

El cuerpo del `CommandLineRunner` contiene la lógica del menú por consola. Se crea un objeto `Scanner` para leer datos del usuario y una variable `opcion` que controla el flujo del programa. Mediante un bucle `do while` el menú se repite hasta que el usuario elige la opción cero. En cada iteración se muestran las distintas opciones disponibles y se lee la elección del usuario.

El `switch` gestiona cada una de las funcionalidades. En la opción de crear personaje se solicitan nombre, clase y nivel, se construye un objeto `Personaje` y se llama al método `crear` del servicio. La llamada está envuelta en un bloque `try catch` para capturar posibles errores, como un nivel inicial inferior a uno. En la opción de listar se llama al método `listar` del servicio y se recorren los resultados mostrando sus datos por pantalla. En la opción de subir nivel se pide el id y la cantidad de niveles a aumentar, y se invoca el método correspondiente del servicio. En la opción de cambiar clase se solicita el id y la nueva clase y se actualiza el personaje. En la opción de eliminar se pide el id y se llama al método `borrar`. Cuando el usuario elige salir, el bucle termina y se muestra un mensaje final. Esta clase representa la capa de presentación, ya que únicamente se encarga de interactuar con el usuario y delega toda la lógica en el servicio.

Personaje.java:

```
package com.example.crudmesa;

import jakarta.persistence.*;

@Entity
public class Personaje {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nombre;
    private String clase;
    private int nivel;

    public Personaje() {}

    public Personaje(String nombre, String clase, int nivel) {
        this.nombre = nombre;
        this.clase = clase;
        this.nivel = nivel;
    }

    public Long getId() { return id; }

    public String getNombre() { return nombre; }

    public String getClase() { return clase; }

    public int getNivel() { return nivel; }

    public void setId(Long id) { this.id = id; }

    public void setNombre(String nombre) { this.nombre = nombre; }

    public void setClase(String clase) { this.clase = clase; }

    public void setNivel(int nivel) { this.nivel = nivel; }
}
```


Personaje.java es la clase que representa el modelo de datos de la aplicación. Está anotada con `@Entity`, lo que indica a JPA que esta clase se corresponde con una tabla de la base de datos. Cada atributo de la clase será una columna de esa tabla. El atributo `id` está anotado con `@Id` para indicar que es la clave primaria, y con `@GeneratedValue` para que su valor se genere automáticamente, utilizando una estrategia de identidad que delega en la base de datos.

La clase contiene los atributos `nombre`, `clase` y `nivel`, que representan las propiedades básicas de cada personaje. Se incluye un constructor vacío obligatorio para que JPA pueda crear instancias de la entidad cuando recupere datos de la base de datos. También se define un constructor con parámetros para facilitar la creación manual de objetos desde el código. Finalmente, se proporcionan los métodos `getters` y `setters` para acceder y modificar los atributos. Esta clase pertenece a la capa de acceso a datos, ya que define la estructura de la información que será persistida.

PersonajeRepository.java:

```
package com.example.crudmesa;

import org.springframework.data.jpa.repository.JpaRepository;

public interface PersonajeRepository
    extends JpaRepository<Personaje, Long> {
}
```

PersonajeRepository.java es la interfaz encargada del acceso directo a la base de datos. Extiende `JpaRepository` indicando como primer parámetro la entidad con la que trabaja, `Personaje`, y como segundo parámetro el tipo de su clave primaria, `Long`. Gracias a esta herencia, Spring Data JPA genera automáticamente la implementación de los métodos básicos de persistencia, como guardar, listar, buscar por `id` o eliminar. No es necesario escribir código SQL ni implementar la interfaz manualmente, ya que Spring se encarga de todo en tiempo de ejecución. Este componente forma parte de la capa de acceso a datos y permite aislar la lógica de persistencia del resto de la aplicación.

PersonajeService.java:

```
package com.example.crudmesa;

import java.util.List;

import org.springframework.stereotype.Service;

@Service
public class PersonajeService {

    private final PersonajeRepository repo;

    public PersonajeService(PersonajeRepository repo) {
        this.repo = repo;
    }

    public Personaje crear(Personaje p) {

        if (p.getNivel() < 1) {
            throw new IllegalArgumentException("El nivel inicial no
puede ser menor que 1");
        }

        return repo.save(p);
    }

    public List<Personaje> listar() {
        return repo.findAll();
    }

    public Personaje buscar(Long id) {
        return repo.findById(id)
            .orElseThrow(() -> new RuntimeException("Personaje no
encontrado"));
    }

    public Personaje subirNivel(Long id, int aumento) {

        Personaje p = buscar(id);
```

```

        int nuevoNivel = p.getNivel() + aumento;

        if (nuevoNivel > 100) {
            nuevoNivel = 100;
        }

        p.setNivel(nuevoNivel);

        return repo.save(p);
    }

    public Personaje cambiarClase(Long id, String nuevaClase) {

        Personaje p = buscar(id);

        p.setClase(nuevaClase);

        return repo.save(p);
    }

    public void borrar(Long id) {
        repo.deleteById(id);
    }
}

```

PersonajeService.java es la clase que contiene la lógica de negocio del sistema. Está anotada con `@Service`, lo que indica que es un componente gestionado por Spring y que pertenece a la capa de servicio. En su interior tiene una referencia al repositorio, que se inyecta mediante el constructor. Esto permite que el servicio utilice el repositorio para acceder a la base de datos sin crearlo manualmente.

El método `crear` comprueba primero que el nivel inicial no sea menor que uno. Si lo es, lanza una excepción para impedir que se guarde un personaje inválido. Si la validación es correcta, delega en el repositorio para guardar el objeto. El método `listar` devuelve todos los personajes almacenados llamando a `findAll`. El método `buscar` intenta localizar un personaje por su id y, si no existe, lanza una excepción para evitar trabajar con valores nulos.

El método `subirNivel` obtiene primero el personaje mediante el método `buscar`. Después calcula el nuevo nivel sumando el aumento indicado. Si el resultado supera el máximo permitido, que es cien, se ajusta el valor a ese límite. Finalmente, actualiza el nivel del

personaje y lo guarda nuevamente en la base de datos. El método cambiarClase también utiliza buscar para obtener el personaje y luego modifica su atributo clase antes de guardarlo. El método borrar simplemente delega en el repositorio para eliminar el registro correspondiente.

Capturas de pantalla

```
--- MESA REDONDA ---  
1. Crear personaje  
2. Listar personajes  
3. Subir nivel  
4. Cambiar clase  
5. Eliminar personaje  
0. Abandonar las Tierras Intermedias  
Opción: 1
```

```
--- MESA REDONDA ---  
1. Crear personaje  
2. Listar personajes  
3. Subir nivel  
4. Cambiar clase  
5. Eliminar personaje  
0. Abandonar las Tierras Intermedias  
Opción: 1  
Nombre: Abdul
```

```
--- MESA REDONDA ---  
1. Crear personaje  
2. Listar personajes  
3. Subir nivel  
4. Cambiar clase  
5. Eliminar personaje  
0. Abandonar las Tierras Intermedias  
Opción: 1  
Nombre: Abdul  
Clase: Guerrero  
Nivel inicial: 5
```

```
--- MESA REDONDA ---
1. Crear personaje
2. Listar personajes
3. Subir nivel
4. Cambiar clase
5. Eliminar personaje
0. Abandonar las Tierras Intermedias
Opción: 1
Nombre: Abdul
Clase: Guerrero
Nivel inicial: 5
Hibernate: insert into personaje (clase,nivel,nombre,id) values (?,?,?,default)
Personaje creado correctamente

--- MESA REDONDA ---
1. Crear personaje
2. Listar personajes
3. Subir nivel
4. Cambiar clase
5. Eliminar personaje
0. Abandonar las Tierras Intermedias
Opción: 2

LISTADO:
Hibernate: select p1_0.id,p1_0.clase,p1_0.nivel,p1_0.nombre from personaje p1_0
1 - Abdul - Guerrero - Nivel 5

--- MESA REDONDA ---
1. Crear personaje
2. Listar personajes
3. Subir nivel
4. Cambiar clase
5. Eliminar personaje
0. Abandonar las Tierras Intermedias
Opción: █
```

```
Opción: 2

LISTADO:
Hibernate: select p1_0.id,p1_0.clase,p1_0.nivel,p1_0.nombre from personaje p1_0
1 - Abdul - Guerrero - Nivel 5

--- MESA REDONDA ---
1. Crear personaje
2. Listar personajes
3. Subir nivel
4. Cambiar clase
5. Eliminar personaje
0. Abandonar las Tierras Intermedias
Opción: 3
ID del personaje: 1
Subir niveles: 8
```

```
LISTADO:
Hibernate: select p1_0.id,p1_0.clase,p1_0.nivel,p1_0.nombre from personaje p1_0
1 - Abdul - Guerrero - Nivel 5

--- MESA REDONDA ---
1. Crear personaje
2. Listar personajes
3. Subir nivel
4. Cambiar clase
5. Eliminar personaje
0. Abandonar las Tierras Intermedias
Opción: 3
ID del personaje: 1
Subir niveles: 8
Hibernate: select p1_0.id,p1_0.clase,p1_0.nivel,p1_0.nombre from personaje p1_0 where p1_0.id=?
Hibernate: select p1_0.id,p1_0.clase,p1_0.nivel,p1_0.nombre from personaje p1_0 where p1_0.id=?
Hibernate: update personaje set clase=?,nivel=?,nombre=? where id=?
Nivel actualizado

--- MESA REDONDA ---
1. Crear personaje
2. Listar personajes
3. Subir nivel
4. Cambiar clase
5. Eliminar personaje
0. Abandonar las Tierras Intermedias
Opción: 2

LISTADO:
Hibernate: select p1_0.id,p1_0.clase,p1_0.nivel,p1_0.nombre from personaje p1_0
1 - Abdul - Guerrero - Nivel 13

--- MESA REDONDA ---
1. Crear personaje
2. Listar personajes
3. Subir nivel
4. Cambiar clase
5. Eliminar personaje
0. Abandonar las Tierras Intermedias
Opción: █
```



```
LISTADO:
Hibernate: select p1_0.id,p1_0.clase,p1_0.nivel,p1_0.nombre from personaje p1_0
1 - Abdul - Guerrero - Nivel 13

--- MESA REDONDA ---
1. Crear personaje
2. Listar personajes
3. Subir nivel
4. Cambiar clase
5. Eliminar personaje
0. Abandonar las Tierras Intermedias
Opción: 4
ID del personaje: 1
Nueva clase: Arquero
Hibernate: select p1_0.id,p1_0.clase,p1_0.nivel,p1_0.nombre from personaje p1_0 where p1_0.id=?
Hibernate: select p1_0.id,p1_0.clase,p1_0.nivel,p1_0.nombre from personaje p1_0 where p1_0.id=?
Hibernate: update personaje set clase=?,nivel=?,nombre=? where id=?
Clase cambiada

--- MESA REDONDA ---
1. Crear personaje
2. Listar personajes
3. Subir nivel
4. Cambiar clase
5. Eliminar personaje
0. Abandonar las Tierras Intermedias
Opción: 2

LISTADO:
Hibernate: select p1_0.id,p1_0.clase,p1_0.nivel,p1_0.nombre from personaje p1_0
1 - Abdul - Arquero - Nivel 13

--- MESA REDONDA ---
1. Crear personaje
2. Listar personajes
3. Subir nivel
4. Cambiar clase
5. Eliminar personaje
0. Abandonar las Tierras Intermedias
Opción: █
```

```

5. Eliminar personaje
0. Abandonar las Tierras Intermedias
Opción: 2

LISTADO:
Hibernate: select p1_0.id,p1_0.clase,p1_0.nivel,p1_0.nombre from personaje p1_0
1 - Abdul - Arquero - Nivel 13

--- MESA REDONDA ---
1. Crear personaje
2. Listar personajes
3. Subir nivel
4. Cambiar clase
5. Eliminar personaje
0. Abandonar las Tierras Intermedias
Opción: 5
ID del personaje: 1
Hibernate: select p1_0.id,p1_0.clase,p1_0.nivel,p1_0.nombre from personaje p1_0 where p1_0.id=?
Hibernate: delete from personaje where id=?
Personaje eliminado

--- MESA REDONDA ---
1. Crear personaje
2. Listar personajes
3. Subir nivel
4. Cambiar clase
5. Eliminar personaje
0. Abandonar las Tierras Intermedias
Opción: 2

LISTADO:
Hibernate: select p1_0.id,p1_0.clase,p1_0.nivel,p1_0.nombre from personaje p1_0

--- MESA REDONDA ---
1. Crear personaje
2. Listar personajes
3. Subir nivel
4. Cambiar clase
5. Eliminar personaje
0. Abandonar las Tierras Intermedias
Opción:

```

```

--- MESA REDONDA ---
1. Crear personaje
2. Listar personajes
3. Subir nivel
4. Cambiar clase
5. Eliminar personaje
0. Abandonar las Tierras Intermedias
Opción: 0
Has abandonado las Tierras Intermedias...
2026-02-15T22:40:24.801Z INFO 18160 --- [crudmesa] [ionShutdownHook] j.LocalContainerEntityManagerFactoryBean : Closing JPA EntityManagerFactory for persistence unit 'default'
2026-02-15T22:40:24.807Z INFO 18160 --- [crudmesa] [ionShutdownHook] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Shutdown initiated...
2026-02-15T22:40:24.816Z INFO 18160 --- [crudmesa] [ionShutdownHook] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Shutdown completed.
PS C:\Users\Asus\Desktop\actut5\crudmesa>

```

Código completo

CrudmesaApplication.java:

```
package com.example.crudmesa;

import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Bean;

import java.util.Scanner;

@SpringBootApplication
public class CrudmesaApplication {

    public static void main(String[] args) {
        SpringApplication.run(CrudmesaApplication.class, args);
    }

    @Bean
    CommandLineRunner ejecutar(PersonajeService service) {

        return args -> {

            Scanner sc = new Scanner(System.in);
            int opcion;

            do {
                System.out.println("\n--- MESA REDONDA ---");
                System.out.println("1. Crear personaje");
                System.out.println("2. Listar personajes");
                System.out.println("3. Subir nivel");
                System.out.println("4. Cambiar clase");
                System.out.println("5. Eliminar personaje");
                System.out.println("0. Abandonar las Tierras Intermedias");
                System.out.print("Opción: ");

                opcion = sc.nextInt();
                sc.nextLine();
            } while (opcion < 0 || opcion > 5);
        }
    }
}
```

```

switch (opcion) {

    case 1 -> {
        System.out.print("Nombre: ");
        String nombre = sc.nextLine();

        System.out.print("Clase: ");
        String clase = sc.nextLine();

        System.out.print("Nivel inicial: ");
        int nivel = sc.nextInt();
        sc.nextLine();

        try {
            service.crear(new Personaje(nombre, clase,
nivel));

            System.out.println("Personaje creado
correctamente");

        } catch (Exception e) {
            System.out.println("Error: " +
e.getMessage());
        }
    }

    case 2 -> {
        System.out.println("\nLISTADO:");
        service.listar().forEach(p ->
            System.out.println(
                p.getId() + " - " +
                p.getNombre() + " - " +
                p.getClase() + " - Nivel " +
                p.getNivel()

            )
        );
    }

    case 3 -> {
        System.out.print("ID del personaje: ");
        Long id = sc.nextLong();

        System.out.print("Subir niveles: ");
        int subida = sc.nextInt();
        sc.nextLine();
    }
}

```

```

        try {
            service.subirNivel(id, subida);
            System.out.println("Nivel actualizado");
        } catch (Exception e) {
            System.out.println("Error: " +
e.getMessage());
        }
    }

    case 4 -> {
        System.out.print("ID del personaje: ");
        Long id = sc.nextLong();
        sc.nextLine();

        System.out.print("Nueva clase: ");
        String nuevaClase = sc.nextLine();

        try {
            service.cambiarClase(id, nuevaClase);
            System.out.println("Clase cambiada");
        } catch (Exception e) {
            System.out.println("Error: " +
e.getMessage());
        }
    }

    case 5 -> {
        System.out.print("ID del personaje: ");
        Long id = sc.nextLong();
        sc.nextLine();

        service.borrar(id);
        System.out.println("Personaje eliminado");
    }
}

} while (opcion != 0);

        System.out.println("Has abandonado las Tierras
Intermedias...");
    };
}

```

}

Personaje.java:

```

package com.example.crudmesa;

import jakarta.persistence.*;

@Entity
public class Personaje {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nombre;
    private String clase;
    private int nivel;

    public Personaje() {}

    public Personaje(String nombre, String clase, int nivel) {
        this.nombre = nombre;
        this.clase = clase;
        this.nivel = nivel;
    }

    public Long getId() { return id; }

    public String getNombre() { return nombre; }

    public String getClase() { return clase; }

    public int getNivel() { return nivel; }

    public void setId(Long id) { this.id = id; }

    public void setNombre(String nombre) { this.nombre = nombre; }

    public void setClase(String clase) { this.clase = clase; }

```

```
public void setNivel(int nivel) { this.nivel = nivel; }  
}
```

PersonajeRepository.java:

```
package com.example.crudmesa;  
  
import org.springframework.data.jpa.repository.JpaRepository;  
  
public interface PersonajeRepository  
    extends JpaRepository<Personaje, Long> {  
}
```

PersonajeService.java:

```
package com.example.crudmesa;  
  
import java.util.List;  
  
import org.springframework.stereotype.Service;  
  
@Service  
public class PersonajeService {  
  
    private final PersonajeRepository repo;  
  
    public PersonajeService(PersonajeRepository repo) {  
        this.repo = repo;  
    }  
  
    public Personaje crear(Personaje p) {  
  
        if (p.getNivel() < 1) {  
            throw new IllegalArgumentException("El nivel inicial no  
puede ser menor que 1");  
        }  
  
        return repo.save(p);  
    }  
}
```

```
public List<Personaje> listar() {
    return repo.findAll();
}

public Personaje buscar(Long id) {
    return repo.findById(id)
        .orElseThrow(() -> new RuntimeException("Personaje no
encontrado"));
}

public Personaje subirNivel(Long id, int aumento) {

    Personaje p = buscar(id);

    int nuevoNivel = p.getNivel() + aumento;

    if (nuevoNivel > 100) {
        nuevoNivel = 100;
    }

    p.setNivel(nuevoNivel);

    return repo.save(p);
}

public Personaje cambiarClase(Long id, String nuevaClase) {

    Personaje p = buscar(id);

    p.setClase(nuevaClase);

    return repo.save(p);
}

public void borrar(Long id) {
    repo.deleteById(id);
}
}
```


application.properties:

```
spring.application.name=crudmesa

spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

spring.jpa.hibernate.ddl-auto=create
spring.jpa.show-sql=true
spring.h2.console.enabled=true
```

pom.xml:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
  http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>4.0.2</version>
    <relativePath/> <!-- lookup parent from repository -->
  </parent>
  <groupId>com.example</groupId>
  <artifactId>crudmesa</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <name>crudmesa</name>
  <description>Demo project for Spring Boot</description>
  <url/>
  <licenses>
    <license/>
  </licenses>
  <developers>
    <developer/>
  </developers>
  <scm>
    <connection/>
    <developerConnection/>
    <tag/>
```

```

        <url/>
    </scm>
    <properties>
        <java.version>17</java.version>
    </properties>
    <dependencies>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-actuator</artifactId>
        </dependency>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-h2console</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>

        <dependency>
            <groupId>com.h2database</groupId>
            <artifactId>h2</artifactId>
            <scope>runtime</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa-test</artifactId>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>
</project>

```

Acceso al zip del proyecto

<https://github.com/RafaelMayor/AED/tree/main/actut5>